FraudLens

Agentic Fraud Investigation System | Production ML + LangGraph Agent + Serverless AWS

Adithya Raj | adithyaraj0604@gmail.com | github.com/Adithya-Raj0604/FraudLens | Live: d30g6wz74gv38k.cloudfront.net

Transactions	Recall (fraud)	Precision	F1 Score	Cost / month	Per investigation
6.3M	**89.9%**	**94.4%**	**92.1%**	**~$0.47**	**~0.5¢**
PaySim dataset	production model	production model	clean feature set	AWS idle	LLM inference

| THE KEY ENGINEERING DECISION

Initial XGBoost model achieved near-perfect metrics — **Recall 99.6%, F1 99.8%** — which was the first red flag, not a win.

- **Root cause diagnosed:** PaySim's fraud simulation sets *newbalanceOrig=0* and leaves *newbalanceDest* unchanged for every fraud transaction — making two engineered features (*orig_zero_after*, *dest_unchanged*) direct encodings of the label.
- **Resolution:** Retrained on a clean feature set excluding leakage flags. Both runs logged side-by-side in MLflow for reproducible, auditable comparison.
- **Honest production metrics:** Recall 89.9%, Precision 94.4%, F1 92.1% — 88 false positives and 166 false negatives across 554K test transactions.

| MODEL COMPARISON (MLflow side-by-side)

Metric	With Leakage	Production (Clean)
Recall (fraud)	99.6%	89.9%
Precision	100%	94.4%
F1 Score	99.8%	92.1%
PR-AUC	0.996	0.971
False Positives	0	88
False Negatives	6	166
F1 @ default 0.5 threshold	0.987	0.621 → 0.921*

* F1 jumps 0.621 → 0.921 after threshold tuning to 0.983 on the precision-recall curve

| COMPLIANCE & SECURITY VALIDATION

- Built FAISS RAG pipeline over **25 OSFI/FINTRAC regulatory documents** — citation-grounded outputs validated against Canadian financial data-privacy standards.
- Resolved 2 undocumented AWS production bugs: misleading 403 on Function URL policy and POST-body signing failure on CloudFront OAC — both fixed via targeted regression tests.
- Shipped **7 pytest regression tests** covering API schema, fraud-score thresholds, and SSE streaming — integrated into Docker CI with CloudWatch 14-day log retention.

| HOW THE AGENT WORKS

1	**Ingest**	6.3M PaySim transactions Dask chunked pipeline
2	**Feature Eng.**	10 features incl. velocity windows + balance diffs
3	**XGBoost+SMOTE**	Train on 2.7M TRANSFER + CASH_OUT rows
4	**LangGraph Agent**	ReAct loop: selects tools based on risk score
5	**Tools**	fraud_score · SHAP explain velocity check · RAG regs
6	**SSE Stream**	Live tool trace to React frontend via Lambda URL

| ARCHITECTURE

Browser	HTTPS	CloudFront
CloudFront	→ /	S3 React app
CloudFront	→ /api	Lambda URL (RESPONSE_STREAM)
Lambda	runs	FastAPI + Lambda Web Adapter
Lambda	loads	XGBoost · SHAP · FAISS · Claude Haiku
API key	stored	SSM Parameter Store (SecureString)
Warm ping	every 5min	EventBridge → /health

| TECH STACK

Data: Pandas (chunked) · Dask · NumPy · PyArrow
ML: XGBoost 3.2 · Scikit-learn · SMOTE
Explainability: SHAP TreeExplainer · Matplotlib
MLOps: MLflow 3.13 (params · metrics · registry)
Agent: LangGraph 1.2 · LangChain · Claude Haiku 4.5
RAG: FAISS 1.14 · sentence-transformers
API: FastAPI · Pydantic v2 · SSE · pytest
Frontend: React 18 · Vite · Tailwind · Recharts
Cloud: Lambda · CloudFront · S3 · ECR · SSM · EventBridge

| COST

~$0.47/month idle (ECR image storage dominates) | **~0.5¢ per investigation** after 4x prompt optimization cut token cost